

상품 데이터 적재 아키텍처 설계서

[TBD] 로 표시된 항목은 아직 결정되지 않은 부분이다. 대화를 통해 하나씩 채워나간다.

0. 문서 작성 규칙

- 이미 코드/대화로 확정된 내용은 그대로 기술한다.
- 아직 결정되지 않은 항목은 [TBD] 로 표시하고, 선택지가 있으면 후보를 함께 적는다.
- 결정되는 즉시 이 문서를 갱신한다.

1. 문서 개요

1.1 목적

`purchase-research-agent` (Python 크롤러)가 ABC마트에서 수집한 상품/리뷰/옵션 데이터를, `purchase-research-backend` (Spring Boot)의 API를 통해 DB에 적재하는 구조를 정의하고, 이 과정에서 예상되는 문제를 정리한다.

1.2 배경

- 기존에 `purchase-research-backend` 스캐폴딩 시점에는 "Spring이 메인이고, FastAPI(Python)를 내부 크롤링 서비스로 호출하며, DB 없이 프록시/가공만 한다"로 결정했었다.
- 이번 요청은 그 반대 방향이다: **Python 에이전트가 크롤링을 수행하고, 결과를 Spring Boot API에 전송(push)하여 DB에 적재한다.**
- 따라서 `purchase-research-backend`에는 영속성 계층(JPA/DB 연동)이 새로 필요하다. 기존 스캐폴딩 (`CrawlerServiceClientConfig` 등 FastAPI를 "호출"하는 구조)은 이번 방향과 맞지 않으므로 재검토가 필요하다. [TBD: 기존 `RestClient` 기반 스캐폴딩을 제거/재활용할지 여부]

1.3 범위

포함:

- ABC마트 상품/리뷰/옵션 데이터의 DB 스키마(DDL) 설계
- Python → Spring Boot 적재 API 명세
- 적재 과정에서의 정합성/중복/오류 처리 정책
- 사용자 요청(자연어) → 조건 조회 → (부족 시 크롤링 트리거) → AI 추천 → 결과 노출까지의 전체 흐름
- 검색어/카테고리 단위 크롤링 대상을 저장해두고 주기적으로 재사용하는 구조 (`crawl_target`)

제외 (현재 단계):

- 실제 결제/구매 연동 — 상품 기본정보·이미지·원본 구매 링크까지만 제공하고, 실제 구매는 사용자가 해당 링크로 이동해서 진행
- ABC마트 외 다른 쇼핑몰
- 프론트엔드 UI (1차는 CLI로 구현, 프론트는 이후 단계)

2. 현재 시스템 현황

2.1 purchase-research-agent (Python / FastAPI / crawl4ai)

- 크롤링 대상: ABC마트 단일 사이트
- 키워드 검색: `AbcMartCrawler.crawl()` — `abcmart.a-rt.com/display/search-word/result`
- 카테고리 검색: `AbcMartCrawler.crawl_category()` — `abcmart.a-rt.com/display/category/main`

- 수집 필드 (리스트 페이지, [crawler.py](#)):
- `title`, `brand`, `price`, `price_original`, `discount_percent`, `image_url`, `color`, `style_code`, `link`, `site`
- 리뷰/옵션 (상세, [detail_fetcher.py](#)):
- `review_count`, `reviews[]` (`content`, `score`, `date`, `size`)
- `options` (`colors[]`, `sizes[]`)
- ABC마트 내부 API(`get-review-list`, `get-prd-option`)를 직접 호출하는 방식, 상위 N개 상품에만 적용(`detail_limit`)
- 저장 방식: 현재는 **DB 없음** — `output/raw/*.json` (원본), `output/*.json` (가격 필터링된 정제본), `output/raw_html/*.html` (파싱 전 원본 HTML)로 로컬 파일 저장만 함
- 실행 방식: FastAPI 서버(`/api/v1/search`, `/api/v1/category`)로 노출되어 있고, 직접 스크립트로도 호출 가능

2.2 purchase-research-backend (Spring Boot)

- 현재 스캐폴딩만 존재 ([build.gradle](#), [PurchaseResearchBackendApplication.java](#))
- `spring-boot-starter-webmvc` + `validation` + `springdoc` (Swagger)만 포함, **JPA/DB 의존성 없음**
- `controller/`, `service/`, `dto/` 패키지는 비어 있음
- 이번 방향 전환에 따라 `spring-boot-starter-data-jpa`, DB 드라이버, (선택) Flyway 추가가 필요할 것으로 보임 [TBD]

3. 목표 아키텍처

3.1 전체 흐름

purchase-research-agent (Python) — 아래 5단계 전부 하나의 에이전트/프로세스 안에서 순차 실행



3.1.1 왜 "HTML 받기"와 "분리(파싱)"를 별도 에이전트로 안 쪼갰는가

결정됨: 쪼개지 않는다. 위 1~5단계 전부 `AbcMartCrawler` / `BackendClient` 하나의 Python 프로세스 안에서 순차 실행한다(참고 샘플 문서 [AI 구매 Agent 상품 수집 시스템.pdf](#)의 HTTP Worker / Parsing Pipeline처럼 물리적으로 분리된 별도 서비스로 두지 않음).

이유:

- "분리해서 얻는 이득"(파싱 로직에 버그가 있어도 재크롤링 없이 재처리 가능)은 이미 1단계에서 원본 HTML을 `output/raw_html/`에 저장해두는 것만으로 확보돼 있다. 굳이 별도 프로세스/큐로 나누지 않아도 재현·재처리가 가능하다.
- 지금은 사이트 하나(ABC마트), 소규모 트래픽이라 물리적으로 나뉘서 얻는 이득(독립적인 스케일링)보다 큐/직렬화 인프라를 관리하는

복잡도가 더 크다.

- 이 판단은 §11 확장성 설계와 별개다 — §11은 "사이트별 로직을 어댑터로 격리"하는 것(같은 프로세스 안에서의 코드 구조)이고, 여기서 얘기하는 건 "fetch와 parse를 다른 프로세스/서비스로 물리적으로 쪼갤지"이다.
- 나중에 사이트가 여러 개로 늘고 크롤링 물량이 커지면(Worker Pool, Queue 도입 등) 그때 재검토한다. [TBD: 재검토 트리거 기준]

3.2 기존 계획 대비 변경 사항

항목	기존 결정 (스케폴딩 시점)	이번 요청
주도 서비스	Spring이 메인, FastAPI를 내부 호출	Python이 크롤링·전송을 주도(push), Spring은 수신·적재
DB	없음 (프록시만)	있음 (상품/리뷰/오피션 영속화)
통신 방향	Spring → FastAPI (요청 시마다 pull)	Python → Spring (크롤링 후 push)

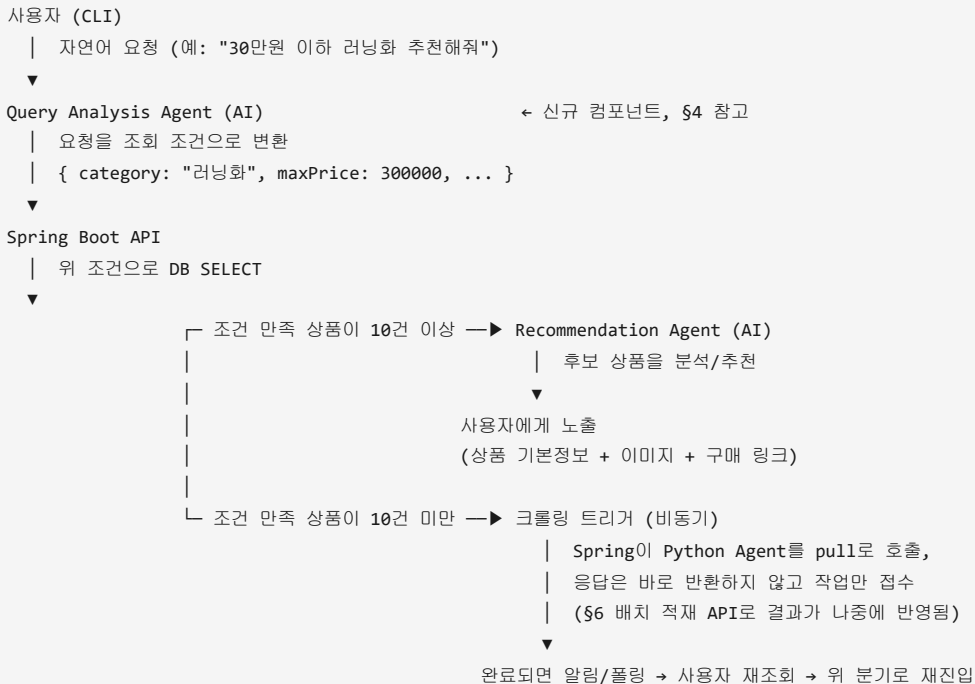
결정됨: 하이브리드로 둘 다 유지한다. 사용자가 즉석 검색을 요청하면(§3.3의 "임계값 미달 시 크롤링 트리거") Spring이 Python을 pull로 호출하고, 배치성 대량 수집·정기 재크롤링(§7.4 `crawl_target`)은 Python이 결과를 push한다. 기존

`CrawlerServiceClientConfig` (RestClient) 스케폴딩은 그대로 pull 경로에 활용한다.

3.3 사용자 요청 → 추천 흐름

구현 상태: 1차 구현 완료 (Query/Recommendation Agent는 규칙 기반 임시 구현). CLI Client(`cli.py`), Spring 조회 API(`GET /api/v1/products/search`), 임계값(10건) 판단, 크롤링 트리거(`POST /api/v1/products/crawl-trigger` , pull 방식), 규칙 기반 Query Analysis Agent(`query_parser.py`)/Recommendation Agent(`recommender.py`) 전부 구현하고 실제로 두 시나리오(임계값 이상/미만)를 동작시켜 확인했다. **다만 Query/Recommendation Agent는 원래 설계인 LLM 호출이 아니라 규칙 기반(정규식/정렬)으로 대체 구현된 상태다** — 이 환경에 `ANTHROPIC_API_KEY` 가 없어서 내린 결정(§12 결정 항목 참고). 비동기 완료 알림(폴링 등)은 아직 없고 CLI가 동기로 기다린다. 상세는 §10.4 참고.

지금까지의 3.1은 "크롤링 → DB 적재"(수집 파이프라인)만 다뤘다. 이 절은 그 위에서 동작하는 "사용자 요청 → 상품 노출"(조회/추천 파이프라인)을 정의한다. 1차는 **CLI**로 구현하고, 이후 프론트엔드로 옮긴다.



검증 메모 (갱신됨):

- 임계값은 **10건**으로 결정했다. [TBD] 카테고리/키워드에 따라 임계값을 다르게 둘 필요가 있는지는 운영해보면서 조정
- 크롤링 트리거는 **처음부터 비동기**로 설계하기로 결정했다(CLI 단계부터). 즉 사용자가 크롤링이 끝날 때까지 터미널을 막고 기다리지

않고, 작업을 접수받았다는 응답을 먼저 주고 완료되면 알림(또는 사용자가 다시 조회)하는 구조다. [TBD] CLI에서 "완료 알림"을 어떻게 구현할지(폴링 명령/파일 기반 상태 확인 등 — 아직 프론트가 없으므로 CLI에 맞는 가벼운 방식 필요)

- Query Analysis Agent와 Recommendation Agent는 **분리된 두 단계**로 구현하기로 결정했다(\$4).
- 결제는 이 흐름에 포함하지 않는다. 사용자는 노출된 `product.link` 로 직접 이동해서 구매한다.

4. 컴포넌트별 책임

컴포넌트	책임
CLI Client	사용자 요청 입력 (1차), 결과(상품 목록) 출력
Query Analysis Agent (AI)	자연어 요청 → 조회 조건(카테고리/가격범위/키워드 등) 변환. Recommendation Agent와 분리된 별도 호출 로 구현 (결정됨) [TBD: 어느 프로세스/서비스에 둘지]
Spring Boot API	조회 조건 기반 DB SELECT(임계값 10건 기준 판단), 크롤링 트리거를 비동기로 호출, 적재 요청 수신, 스키마/유효성 검증, 중복 판별, DB 적재
Recommendation Agent (AI)	SELECT된 후보 상품을 분석해 사용자에게 추천. Query Analysis Agent와 별도 호출(결정됨)
Python Agent (crawler)	ABC마트 크롤링, 리뷰/옵션 수집, 1차 정제(가격 유무 필터 등), Spring API 호출(적재), 크롤링 트리거 수신 시 실행
DB	상품/옵션/리뷰/가격이력/크롤링 대상(<code>crawl_target</code>) 영속 저장

5. 데이터베이스 설계 (DDL)

5.1 테이블 목록 (초안)

테이블	설명	상태
<code>product</code>	상품 기본 정보 (title, brand, price 등)	초안
<code>price_history</code>	가격 변동 이력	결정됨
<code>product_option</code>	색상/사이즈 옵션	초안
<code>product_review</code>	리뷰	초안
<code>crawl_target</code>	주기적으로 재크롤링할 검색어/카테고리 단위 크롤링 대상(URL 등)	신규 추가, \$7.4 참고
<code>crawl_batch</code>	수집 배치 이력(언제/무슨 키워드로 몇 개 수집했는지)	[TBD: 필요 여부]

5.2 DDL 초안 (MySQL)

```
-- 상품 기본 정보 (가격의 "현재값"만 가짐 - 이력은 price_history에 별도 적재)
CREATE TABLE product (
  id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  site        VARCHAR(20)     NOT NULL,             -- 'abcmart' 등 사이트 식별자 (향후 다른 사이트 추가 대비)
  source_product_id VARCHAR(50) NOT NULL,           -- 사이트 내부 상품 ID (ABC마트는 prdtNo). 필드명을 사이트 종속적이
  지 않게 유지
  title       VARCHAR(300)    NOT NULL,
  brand       VARCHAR(100),
  price       INT,             -- 원 단위 정수로 저장 (문자열 "35,000원" -> 파싱)
  price_original INT,
  discount_percent INT,
  image_url   VARCHAR(500),
  style_code  VARCHAR(50),
  link        VARCHAR(500)    NOT NULL,
  review_count INT           NOT NULL DEFAULT 0,
  collected_at DATETIME       NOT NULL,
  created_at  DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_product_site_source_id (site, source_product_id) -- [TBD: 이 조합이 정말 유니크한 상품 식별자인가?]
);

-- 가격 이력 (재수집 때마다 새 row를 INSERT, product.price는 최신값으로만 갱신)
CREATE TABLE price_history (
  id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  product_id  BIGINT          NOT NULL,
  price       INT             NOT NULL,
  price_original INT,
  discount_percent INT,
  collected_at DATETIME       NOT NULL,
  CONSTRAINT fk_price_history_product FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE CASCADE,
  KEY idx_price_history_product_collected (product_id, collected_at)
);

-- 상품 옵션 (색상/사이즈)
CREATE TABLE product_option (
  id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  product_id  BIGINT          NOT NULL,
  option_type VARCHAR(20)     NOT NULL,             -- 'COLOR' | 'SIZE'
  option_value VARCHAR(50)    NOT NULL,
  CONSTRAINT fk_product_option_product FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE CASCADE
  -- [TBD: UNIQUE(product_id, option_type, option_value) 걸 것인지]
);

-- 상품 리뷰
-- 주의: 작성자 식별 정보(닉네임/회원ID 등)는 수집하지 않는다. review_source_id는 리뷰 자체의
-- 일련번호(예: ABC마트 rvwId)일 뿐 개인정보가 아니며, 재수집 시 중복 판별 용도로만 쓴다.
CREATE TABLE product_review (
  id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  product_id  BIGINT          NOT NULL,
  review_source_id VARCHAR(50) NOT NULL,           -- ABC마트 리뷰 일련번호(rvwId 등) - 크롤러에서 추가 파싱 필요($9)
  content     TEXT,
  score       DECIMAL(3,1),
  review_date DATE,
  size        VARCHAR(20),
  created_at  DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_review_product FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE CASCADE,
```

```
    UNIQUE KEY uk_product_review_source (product_id, review_source_id)
);
```

매 재수집 시 처리 순서(결정됨): ① `product` row가 있으면 `price / price_original / discount_percent / review_count / updated_at` 만 UPDATE, 없으면 INSERT ② 항상 `price_history` 에 새 row INSERT.

```
-- 주기적으로 재크롤링할 대상 (검색어/카테고리 단위 - 개별 상품 링크(product.link)와는 별개)
CREATE TABLE crawl_target (
    id                BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
    site              VARCHAR(20)     NOT NULL DEFAULT 'abcmart',
    target_type       VARCHAR(20)     NOT NULL,                -- 'KEYWORD' | 'CATEGORY'
    keyword_or_category VARCHAR(100)  NOT NULL,                -- 예: '구두' 또는 '스니커즈_남성'
    request_url       VARCHAR(500),    -- 실제로 재사용할 크롤링 요청 URL (또는 URL 템플릿)
    max_items         INT             NOT NULL DEFAULT 200,
    recrawl_interval_minutes INT       NOT NULL,                -- [TBD: 기본값/카테고리별 차등 여부]
    last_crawled_at   DATETIME,
    enabled           BOOLEAN          NOT NULL DEFAULT true,
    created_at        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uk_crawl_target (site, target_type, keyword_or_category)
);
```

5.3 미정 사항

- [TBD] PK 전략: `AUTO_INCREMENT` vs UUID
- [TBD] 인덱스 설계 (검색 키워드로 조회할 일이 있는지, brand/price 범위 검색이 필요한지)

결정됨: 리뷰 원본 ID(작성자 식별 정보 아님, 리뷰 자체의 일련번호)를 크롤러에서 추가로 파싱해서 `review_source_id` 로 저장한다(위 DDL 반영 완료). 크롤러 쪽 작업 필요 — `detail_fetcher.py` 가 현재 버리고 있는 `rvwId` (또는 동등 필드)를 추가로 담아야 함. [TBD: 실제 코드 반영은 별도 작업으로 진행]

결정됨: DB는 **MySQL**로 한다. (`agentpay-guard-api-server` 의 PostgreSQL 구성과는 무관 — 이 프로젝트는 별도로 신경 쓰지 않는다.) 가격 정보는 **가격 이력 테이블(`price_history`)을 별도로 유지**한다(\$5.2, \$7.2 갱신 반영).

6. Python → Spring API 연동 명세

6.1 엔드포인트

결정됨: 배치 전송. `POST /api/v1/products/batch` (수집된 상품 리스트를 한 번에 전송). 크롤링 1회당 수십~수백 건이 나오므로 건별 전송보다 배치가 유리하다고 판단.

결정됨: 배치 1건 최대 **100건**으로 제한한다. Python Agent가 크롤링 결과가 100건을 넘으면 100건 단위로 쪼개서 여러 번 호출한다(예: 300건 수집 시 3번 호출). [TBD] 분할 호출 중 일부 청크만 실패했을 때 나머지 청크는 계속 보낼지 중단할지

6.2 요청/응답 포맷 [TBD]

```
// 요청 예시 (초안, 확정 아님)
{
  "site": "abcmart",
  "keyword": "구두",
  "collectedAt": "2026-07-26T01:17:34+09:00",
  "products": [
    {
      "sourceProductId": "1010110646",
      "title": "런너 러너 3",
      "brand": "뉴발란스",
      "price": 19000,
      "priceOriginal": 49000,
      "discountPercent": 61,
      "imageUrl": "https://...",
      "link": "https://abcmart.a-rt.com/product?prdtNo=1010110646",
      "reviewCount": 13,
      "options": { "colors": ["BLACK"], "sizes": ["225", "230"] },
      "reviews": [ { "content": "...", "score": 5, "date": "2026-01-01", "size": "250" } ]
    }
  ]
}
```

6.3 인증 방식

결정됨: 최소한의 API Key를 둔다. 요청 헤더(예: `x-API-Key`)로 검증하는 가장 단순한 방식으로 시작한다. [TBD] 키 값 관리 방식 (환경변수/설정파일), 키 검증을 Spring 어디 계층(Filter/Interceptor)에 돌지는 세부 구현 시 결정

6.4 실패 시 동작

결정됨: 부분 성공을 허용한다(All or Nothing 룰백 아님). 배치 안에서 실패한 상품이 있어도 성공한 상품은 그대로 적재하고, 상품별 성공/실패 결과를 응답에 담아 Python이 실패 건만 별도로 로그를 남기거나 재시도할 수 있게 한다.

```
// 응답 예시 (초안)
{
  "totalCount": 30,
  "successCount": 28,
  "failCount": 2,
  "failures": [
    { "sourceProductId": "1010110646", "reason": "가격 파싱 실패: price is null" }
  ]
}
```

7. 데이터 정합성 및 중복 처리

7.1 상품 식별 키

- 사이트 내부 상품 ID(ABC마트는 `prdtNo`, 상품 URL의 `prdtNo=` 값)를 `source_product_id` 로 저장하고 이를 기준으로 식별하는 것이 유력. (`site`, `source_product_id`) 유니크 제약.
- 다른 사이트가 추가되면 그 사이트의 내부 상품 ID를 그대로 `source_product_id` 에 넣으면 되므로, 이 컬럼 자체는 사이트에 무관하게 재사용 가능하다.

7.2 재크롤링 시 갱신 전략

결정됨: (c) 방식. `product` 테이블은 최신 가격/할인율/리뷰수로 UPDATE하고, 가격 변동 이력은 `price_history` 테이블에 매 수집마다 INSERT한다. (§5.2 DDL 참고)

- [TBD] 재고(품질 여부) 개념은 현재 크롤러가 수집하고 있지 않음 — `stock_history` 도입은 크롤러 쪽에 품질 판별 로직이 먼저 추가돼야 함(미착수)

7.3 리뷰 중복 처리

결정됨: §5.2/§5.3에서 반영한 `review_source_id` (리뷰 자체의 일련번호, 작성자 식별 정보 아님) 기준 `UNIQUE(product_id, review_source_id)` 제약으로 중복을 막는다. 재수집 시 이미 있는 `review_source_id` 는 INSERT를 스킵(또는 무시)하면 된다. [TBD] 크롤러(`detail_fetcher.py`)에서 이 필드를 실제로 파싱하는 코드 작업은 아직 안 함.

7.4 주기적 재크롤링과 `crawl_target` 재사용

아이디어 검증: "크롤링할 때 입력하는 API 주소(검색어/카테고리 URL)를 저장해뒀다가, 나중에 주기적으로 재크롤링할 때 그대로 재사용하자"는 아이디어는 타당하다. 다만 두 가지 "주소"를 구분해야 한다.

구분	저장 위치	용도	재사용 시점
개별 상품 링크	<code>product.link</code>	상품 1건의 최신 가격/재고 재검증	결제 직전 등 즉시성이 필요할 때
검색어/카테고리 크롤링 대상	<code>crawl_target</code> (신규)	카테고리/키워드 전체를 다시 훑어서 신상품·가격변동을 반영	스케줄러가 주기적으로 실행할 때

동작 방식(초안):

1. Python Agent가 크롤링을 실행할 때마다(§3.3의 크롤링 트리거 포함) 사용한 `site + target_type + keyword_or_category + request_url` 을 `crawl_target` 에 upsert(없으면 INSERT, 있으면 `last_crawled_at` 갱신)한다.
2. 별도의 스케줄러([TBD] — Spring `@Scheduled` 인지, 별도 배치 프로세스인지)가 `crawl_target` 에서 `enabled = true` 이고 `last_crawled_at` 이 `recrawl_interval_minutes` 를 초과한 row를 찾아 Python Agent에게 재크롤링을 요청한다.
3. 이 방식이면 "무엇을 다시 크롤링해야 하는지"를 매번 사람이 정하거나 하드코딩할 필요 없이 DB가 기억하고 있는 셈이 된다.

[TBD] 스케줄러를 Spring에 둘지 Python에 둘지, `recrawl_interval_minutes` 를 카테고리별로 어떻게 정할지(인기 카테고리는 더 자주?)는 아직 결정하지 않았다.

8. 오류 및 재시도 정책 [TBD]

- Python → Spring 호출이 네트워크 오류로 실패했을 때: 재시도할지, 로컬 JSON(`output/`)에 이미 저장돼 있으니 나중에 재전송할지
- Spring이 DB 저장에 실패했을 때: Python에 몇 번 코드로 응답할지, Python은 그 실패를 어떻게 인지하고 처리할지

9. 실제로 겪었거나 예상되는 문제

이번 프로젝트에서 이미 겪었던 것 포함, 이 적재 과정에서 특히 주의해야 할 것들:

문제	설명	대응 방향
크롤링 URL 자체가 조용히 실패할 수 있음	실제로 겪었던 사례: ABC마트 검색 URL이 파라미터명이 틀려서 항상 홈페이지 위젯을 반환했는데, 크롤러는 "성공"으로 보고했음(에러가 안 남)	Spring 적재 API 쪽에서도 최소한의 이상 탐지(가격 0원, 동일 상품이 비정상적으로 반복 등) 필요
콘솔/파일 인코딩	Windows 콘솔(cp949)과 UTF-8 데이터 간 표시 문제를 겪음(실제 데이터는 정상이었음)	Python-Spring 간 전송은 HTTP+JSON(UTF-8)이라 문제 없을 가능성 높지만, 로그 출력 시 유의

문제	설명	대응 방향
외부 API(리뷰/옵션) 호출 실패	<code>detail_fetcher.py</code> 에서 "Server disconnected without sending a response" 에러가 종종 발생(수집 10건 중 1~4건 정도)	이 데이터가 없는 상태로 Spring에 전송될 수 있음 — <code>review_count=0</code> , <code>options={}</code> 인 것이 "실제로 리뷰/옵션이 없다"인지 "수집 실패"인지 구분 필요 [TBD: 필드에 수집 성공 여부 플래그 추가?]
리뷰 중복 적재	위 7.3 참고	<code>review_source_id</code> UNIQUE 제약으로 해결(결정됨). 크롤러 파싱 코드 작업만 남음
Python-Spring 간 스키마 불일치	크롤러 필드가 바뀌면(이미 이번 세션에서 <code>price_original</code> , <code>discount_percent</code> 등 필드를 추가한 이력 있음) Spring DTO와 어긋날 수 있음	필드 추가/변경 시 두 프로젝트를 함께 리뷰하는 프로세스 필요
크롤링 주기와 적재 처리량	대량 크롤링(예: <code>max_items=500</code>) 후 한 번에 push하면 Spring/DB에 순간 부하	배치 크기 제한, 페이지 단위 전송 등 [TBD]
비동기 크롤링 트리거의 구현 복잡도	\$3.3에서 처음부터 비동기로 설계하기로 결정(카테고리 크롤링은 수십 초~분 단위 걸림)	1차 구현은 동기로 우회함 (§10.4) — CLI가 크롤링 끝날 때까지 그냥 기다림. 비동기 알림 방식은 여전히 [TBD] (§12 #18)
조건 매핑 모호성	자연어 조건(가격/카테고리)을 SQL WHERE로 어떻게 매핑할지 미정	1차 구현 완료: <code>query_parser.py</code> 가 정규식으로 가격만 뽑고 나머지 텍스트를 <code>title LIKE</code> 키워드로 그대로 씌움(§10.4). 카테고리 컬럼 자체가 DB에 없어서 "카테고리"가 아니라 "제목에 포함된 단어" 수준의 매칭임
AI 요청 분석 오류	자연어를 잘못 해석하면 엉뚱한 조건으로 SELECT하거나 불필요한 크롤링이 발생할 수 있음	규칙 기반이라 LLM 오해석 리스크는 없지만, 대신 규칙이 못 알아듣는 표현(예: "5만원대", "저렴한 거")은 그냥 키워드로 통째로 들어가서 검색이 안 맞을 수 있음. LLM 교체 시 재검토 필요(§12 #17)

10. 개발 단계

- ✅ DDL 확정 및 `purchase-research-backend` 에 JPA 엔티티/마이그레이션 추가
- ✅ 적재 API(Controller / Service / DTO) 구현
- ✅ Python 쪽에 Spring API 호출 로직 추가 (`app/services/backend_client.py`)
- ✅ end-to-end 테스트 — 로컬에 JDK 21(Temurin) + MySQL 8.4를 직접 설치해서 실제로 확인함(아래 §10.2)
- ✅ 중복/재수집 시나리오 테스트 — 실제 MySQL로 확인함
- [TBD] 실패 처리(네트워크 오류, 부분 실패) 테스트 — 백엔드 연결 자체가 끊긴 경우는 확인(§10.2), 배치 안 특정 상품만 실패하는 경우는 아직 실제로 재현 안 해봄
- ✅ §3.3 조회/추천 파이프라인 1차 구현 (CLI, Spring 조회/트리거 API, 규칙 기반 Query/Recommendation Agent) — 상세는 §10.4
- [TBD] 규칙 기반 Query/Recommendation Agent를 실제 LLM 호출로 교체

10.1 이번에 구현된 파일

purchase-research-agent (Python)

- `app/crawlers/abcmart/detail_fetcher.py` — `review_source_id` (리뷰 일련번호, `prdtRvwSeq`) 파싱 추가, `score` 추출 버그 수정(기존엔 항상 `None`이었음 — 실제 평점은 `productReviewEvlt`s 배열 안의 `prdtRvwCode == "10000"` 항목에 있었음)
- `app/crawlers/abcmart/crawler.py` — 공통 스키마에 `source_product_id` 필드 추가
- `app/services/backend_client.py` — `BackendClient`: 100건 단위 청크 전송, X-API-Key 헤더, ID 없는 리뷰는 전송 제외, 청크 실패 시에도 나머지 계속 진행
- `app/services/query_parser.py` — Query Analysis Agent 임시 구현(규칙 기반): "N만원 이하/미만" 정규식으로 가격 추출, 나머지 텍스트를 키워드로
- `app/services/recommender.py` — Recommendation Agent 임시 구현(규칙 기반): 할인율 내림차순 → 가격 오름차순 정렬 후 상위 5개
- `cli.py` — §3.3 CLI 진입점. 입력 → 분석 → Spring 조회 → (필요시) 크롤링 트리거 → 재조회 → 추천 출력
- `.env` / `.env.example` — `BACKEND_BASE_URL`, `BACKEND_API_KEY` 추가 (`.env` 는 gitignore 대상, 로컬에 `BACKEND_API_KEY=local-dev-key` 로 채워둠)

purchase-research-backend (Spring Boot)

- `build.gradle` — `spring-boot-starter-data-jpa`, `spring-boot-starter-flyway`, `flyway-mysql`, `mysql-connector-j` 추가
- `src/main/resources/application.yaml` — MySQL 데이터소스, JPA(`ddl-auto: validate`), Flyway, `app.security.api-key` (X-API-Key 값, 환경변수 `PRODUCT_INGEST_API_KEY` 로 오버라이드 가능) 추가
- `src/main/resources/db/migration/V1__init_schema.sql` — §5.2 DDL 그대로 반영
(`product/price_history/product_option/product_review/crawl_target`)
- `domain/` — `Product`, `PriceHistory`, `ProductOption`, `ProductReview` JPA 엔티티 (`crawl_target` 은 스케줄러가 아직 없어서 엔티티는 만들지 않고 테이블만 준비)
- `repository/ProductRepository.java` — `findBySiteAndSourceProductId` + `search(keyword, maxPrice)` (JPQL, title LIKE + price 이하, 둘 다 nullable)
- `dto/` — `ProductBatchRequest`, `ProductPayload`, `OptionsPayload`, `ReviewPayload`, `ProductBatchResponse` (+ `FailureItem`), `ProductSearchResponse` (+ `ProductSummary`), `FastApiProductItem`, `FastApiSearchResponse` (FastAPI 응답 `snake_case`를 매핑하는 전용 DTO)
- `security/ApiKeyFilter.java` — `/api/v1/products/**` 경로만 X-API-Key 헤더로 보호하는 서블릿 Filter (Spring Security 없이 최소 구현)
- `service/ProductIngestService.java` — `upsert(product)` + 항상 `INSERT(price_history)` + 옵션 교체 + 리뷰 중복 스킵, 상품 단위 부분 성공 처리
- `service/ProductQueryService.java` — §3.3 "Spring Boot API: DB SELECT" 부분. 순수 조회만 담당(임계값 판단은 CLI 쪽)
- `service/CrawlTriggerService.java` — §3.3 "크롤링 트리거(pull)" 구현체. FastAPI `/api/v1/search` 를 RestClient로 호출하고, 응답을 곧바로 `ProductIngestService.ingest()` 로 넘겨 같은 요청 안에서 적재까지 끝냄(별도 `/batch` 왕복 없음)
- `controller/ProductController.java` — `POST /api/v1/products/batch`, `GET /api/v1/products/search`, `POST /api/v1/products/crawl-trigger`
- `docker-compose.yml` — 로컬 개발용 MySQL 8.4 (루트의 `agentpay-guard` compose와는 완전히 별개)

10.2 실제 검증 내역

이 환경에 원래 Java/MySQL/Docker가 전혀 없었는데, winget으로 **Eclipse Temurin JDK 21**과 **MySQL 8.4**를 직접 설치하고(Docker는 여전히 없어서 `mysqld` 를 수동 초기화 후 로컬 프로세스로 기동), `purchase_research` DB/계정을 만들어서 실제로 끝까지 확인했다.

- `./gradlew compileJava` → **BUILD SUCCESSFUL**
- `./gradlew bootRun` → Flyway가 `V1__init_schema.sql` 을 실제 MySQL에 적용, Hibernate가 JPA 엔티티를 실제 스키마와 대조 검증(`ddl-auto: validate`) 통과, Tomcat이 8081에서 정상 기동
- Python(`AbcMartCrawler.crawl` → `attach_details` → `BackendClient.push_products`)으로 ABC마트 "부츠" 실제 3건을 크롤링해서 진짜 HTTP로 전송 → `{"totalCount": 3, "successCount": 3, "failCount": 0}` 응답
- MySQL에 직접 접속해서 `product` (3행, 한글 필드 정상), `price_history` (3행), `product_option` (색상/사이즈), `product_review` (6행, `score` 가 이제 실제 값으로 들어감)까지 데이터가 정확히 들어간 것을 SQL로 확인
- **재수집 시나리오**: 같은 3건을 다시 크롤링해서 재전송 → `product` 3행 그대로 유지(중복 INSERT 안 됨, UPDATE만 발생, `updated_at` 갱신 확인) / `price_history` 3행 → 6행으로 증가(재수집마다 이력 누적) / `product_review` 6행 그대로 유지 (`review_source_id` 기준 중복 스킵 정상 동작)

10.3 알려진 한계

- `ProductIngestService.ingest()` 가 배치 전체를 하나의 트랜잭션으로 처리한다. 상품 하나가 처리 도중(예: 옵션 저장 단계에서) 실패하면 그 상품에 대해 이미 저장된 `product / price_history` row는 트랜잭션이 롤백되지 않으므로 그대로 남는다. 상품 단위로 완전히 원자적으로 만들려면 별도 트랜잭션 경계(예: `REQUIRES_NEW`)가 필요한데, 이번 범위에서는 넣지 않았다.
- 배치 안에서 "일부 상품만 실패"하는 케이스(§6.4)는 아직 실제로 재현해서 확인하지 못했다 — 정상 케이스와 완전 연결 실패 케이스만 검증됨.
- 지금 로컬 MySQL은 Windows 서비스로 등록하지 않고 `mysqld` 를 직접 백그라운드 프로세스로 띄운 상태라, 재부팅하면 꺼진다. 계속 쓰려면 서비스 등록이 필요하다([TBD]).

10.4 조회/추천 파이프라인(§3.3) 구현 체크리스트

- [x] CLI Client — `cli.py` (`python cli.py` 로 실행, 자연어 입력 → 결과 출력)
- [x] Query Analysis Agent — `app/services/query_parser.py`. 규칙 기반 임시 구현(정규식으로 "N만원 이하/미만" 가격, 나머지 텍스트를 키워드로 추출). LLM 아님

- [x] Spring 조회(SELECT) API — `GET /api/v1/products/search?keyword=&maxPrice=` (`ProductQueryService.java`, title LIKE + price 이하 조건)
- [x] 임계값(10건) 판단 로직 — Spring이 아니라 **CLI(Python) 쪽에 둬** (§16 결정, 아래 참고)
- [x] Recommendation Agent — `app/services/recommender.py`. 규칙 기반 **임시 구현**(할인율 높은 순 → 가격 낮은 순 정렬, 상위 5개). LLM 아님
- [x] Spring → Python 크롤링 트리거(pull) — `POST /api/v1/products/crawl-trigger` (`CrawlTriggerService.java`): FastAPI `/api/v1/search` 를 호출해서 크롤링시키고, 받은 결과를 같은 요청 안에서 바로 `ProductIngestService` 로 적재까지 함(별도 `/batch` 왕복 없음)
- [] 비동기 완료 알림/풀링 (§3.3, §9) — **미구현**. 지금은 CLI가 크롤링 끝날 때까지 동기로 기다림(§12 결정 항목에 사유 정리)
- [] `crawl_target` 스케줄러 (§7.4) — 테이블만 만들어져 있고 아무것도 채우거나 읽지 않음

실제 검증(2026-07-26): `python cli.py` 에 "부츠 추천해줘" 입력 → DB에 부츠 3건뿐이라 임계값 미달 → 크롤링 트리거(성공 30건/실패 0건) → 재조회 14건 → 할인율순 상위 5개 추천 출력까지 확인. 같은 키워드로 재실행하면(이제 14건 ≥ 10) 크롤링 트리거 없이 바로 추천으로 넘어가는 것도 확인.

호출 체인 증거 수준

위 시나리오를 실행했을 때 각 단계가 실제로 일어났다는 걸 뭘 근거로 아는지 구분해서 남긴다. "일어났다고 텍스트가 출력됐다"와 "다른 시스템의 독립적인 기록으로 교차 확인됐다"는 신뢰도가 다르다.

로그/DB로 직접 증명된 단계:

단계	내용	증거
Spring → FastAPI 요청	<code>CrawlTriggerService.triggerAndIngest()</code> 가 <code>crawlerServiceRestClient.post().uri("/api/v1/search"...)</code> 로 실제 호출	FastAPI 서버 자체 접속 로그: <code>POST /api/v1/search?keyword=%EB%B6%80%EC%B8%A0&site=abcmart&max_items=30 HTTP/1.1" 200 OK</code>
FastAPI가 실제 ABC마트를 크롤링	<code>crawler.py</code> 의 <code>AbcMartCrawler.crawl()</code>	<code>[FETCH]... https://abcmart.a-rt.com/...</code> , <code>[ABCMART:search] page=1 +30개</code> , <code>[RAW] 30개 -> output/raw/abcmart_부츠_raw_20260726_192112.json</code> (실제 파일 존재)
FastAPI → Spring 응답 성공	위와 같은 요청의 응답	같은 로그 라인의 <code>200 OK</code>
Spring → MySQL 적재	<code>ProductIngestService.ingest()</code>	MySQL 직접 SELECT로 확인: 상품 14건, <code>updated_at = 2026-07-26 19:21:25</code> (요청 시각과 일치), <code>price_history</code> 20행 누적

로그는 없지만 코드 흐름상 확실한 단계 (별도 프로세스 간 호출이 아니라서 독립 로그가 안 남음):

단계	왜 로그가 없나
<code>cli.py</code> → Spring <code>GET /search</code> , <code>POST /crawl-trigger</code> 호출	Spring에 요청 접근 로그(access log)를 별도로 설정 안 함 — Spring이 요청을 받았다는 걸 Spring 쪽 로그로는 못 봄
FastAPI 내부 <code>search.py</code> → <code>crawler_service.py</code> 함수 호출	같은 파이썬 프로세스 안의 함수 호출이라 별도 로그 없음
Spring <code>CrawlTriggerService</code> → <code>ProductIngestService.ingest()</code>	같은 JVM 안의 자바 메서드 직접 호출(HTTP 아님)이라 로그 없음. 결과물(MySQL 적재)로 간접 증명됨
Spring → <code>cli.py</code> 최종 응답	<code>cli.py</code> 가 콘솔에 찍은 텍스트뿐이라, 그 자체의 독립 증거는 아님(다만 그 다음 재조회에서 <code>totalCount</code> 가 실제로 늘어난 것이 간접 증거)

[TBD] Spring에 요청 로깅을 추가하면 위 "로그 없는 단계"도 직접 증명 가능해짐 — 지금은 우선순위가 낮아서 안 함.

11. 확장성 설계 (다중 사이트 지원)

이 프로젝트는 ABC마트 하나로 시작하지만, 사이트가 추가되거나 필드가 바뀔 때 핵심 로직을 건드리지 않고 확장할 수 있어야 한다. 참고한 샘플 문서(AI 구매 Agent 상품 수집 시스템.pdf \$5.8 Site Adapter)의 패턴을 이 프로젝트 규모에 맞게 적용한다.

11.1 원칙

1. **사이트별 로직은 어댑터로 격리한다.** 공통 크롤링 흐름(요청 → 파싱 → 정제 → 저장)은 그대로 두고, 사이트마다 다른 부분(URL, CSS 셀렉터, API 응답 구조)만 사이트별 모듈에 둔다.
2. **공통 상품 데이터 스키마를 하나로 유지한다.** 지금 `_parse()` 가 만드는 `title/brand/price/price_original/discount_percent/image_url/color/style_code/link/site` 필드셋이 사실상 공통 스키마 역할을 하고 있다. 새 사이트를 추가해도 이 필드 구조를 그대로 채우도록 만든다 (사이트마다 있는 필드가 달라도, 없는 값은 빈 값/None으로 채움).
3. **DB 스키마는 이미 사이트 중립적으로 설계했다.** `product.site`, `source_product_id` (§7.1에서 방금 이름 변경), `crawl_target.site` — 전부 사이트를 값으로 받는 구조라 테이블 자체는 사이트가 늘어도 그대로 쓸 수 있다.
4. **사이트 등록은 설정으로, 로직 추가는 어댑터로.** 새 사이트를 켜고 끄는 것과 새 사이트의 파싱 로직을 구현하는 것을 분리한다.

11.2 현재 코드 구조 (반영 완료)

아래 리팩터링을 실제로 진행했다.

이전	변경 후
<code>crawler_service.py</code> 에 <code>if site == "abcmart": ... else: [], []</code>	<code>app/crawlers/__init__.py</code> 의 <code>SITE_CRAWLERS: dict[str, type[SiteCrawler]]</code> 레지스트리를 조회하는 방식으로 대체. 새 사이트는 이 dict에 한 줄 추가하면 됨
<code>abcmart.py</code> 하나에 URL 상수, 파싱, 크롤링이 전부 들어있음	<code>base.py</code> 에 <code>SiteCrawler</code> 추상 인터페이스(<code>crawl</code> , <code>crawl_category</code> , <code>attach_details</code>) 정의, <code>AbcMartCrawler</code> 가 이를 구현하도록 정리. <code>crawlers/abcmart/</code> 패키지로 분리(<code>crawler.py</code> , <code>detail_fetcher.py</code> , <code>__init__.py</code>) — 다음 사이트는 <code>crawlers/{new_site}/</code> 에 같은 형태로 추가
<code>detail_fetcher.py</code> 가 최상위에 단독으로 있었음	<code>crawlers/abcmart/detail_fetcher.py</code> 로 이동, <code>AbcMartCrawler.attach_details()</code> 를 통해 <code>SiteCrawler</code> 인터페이스로 노출
<code>SUPPORTED_SITES = ["abcmart"]</code> 하드코딩	<code>SUPPORTED_SITES = list(SITE_CRAWLERS.keys())</code> 로 레지스트리에서 자동 파생

현재 구조:

```
app/crawlers/
  __init__.py      # SITE_CRAWLERS 레지스트리
  base.py          # SiteCrawler 추상 인터페이스
  abcmart/
    __init__.py
    crawler.py      # AbcMartCrawler(SiteCrawler)
    detail_fetcher.py # DetailFetcher (ABC마트 리뷰/옵션 API)
```

`search_by_category` (카테고리 수집)는 `CATEGORIES` 자체가 ABC마트 카테고리 체계라 아직 `site` 파라미터 없이 ABC마트로 고정돼 있다. [TBD] 다른 사이트가 카테고리 수집을 지원하게 되면 이 부분도 `site` 파라미터를 받도록 확장 필요.

11.3 신규 사이트 추가 시 체크리스트 (초안)

1. 사이트 접근 정책 확인 (robots.txt, 이용약관)
2. 검색/카테고리 크롤링 URL 및 파라미터 분석 — 이번엔 ABC마트에서 겪었듯, 실제 브라우저로 검색을 실행해서 나오는 요청을 직접 확인해야 한다(추측한 URL은 틀릴 수 있음, §9 참고)
3. 상품 리스트 HTML/JSON 구조 분석 → 공통 스키마로 매핑
4. 리뷰/옵션 수집 방식 분석 (API가 있는지, 없으면 상세페이지 파싱이 필요한지)
5. 사이트별 어댑터 구현 + 크롤러 레지스트리에 등록
6. `crawl_target` 에 해당 사이트의 검색어/카테고리 등록

12. 미결정 사항 정리 (Open Questions)

결정됨

1. ~~DB 종류~~ → **MySQL** (agentpay-guard-api-server 의 PostgreSQL 구성과는 무관)
2. ~~배치 vs 건별 전송~~ → **배치 전송** (POST /api/v1/products/batch)
3. ~~재수집 시 가격 처리~~ → **product UPDATE + price_history 별도 이력 테이블**
4. ~~ SiteCrawler 리팩터링 시점~~ → **바로 진행, 완료됨** (\$11.2). 실제 크롤링(키워드/카테고리+상세)과 FastAPI HTTP 엔드포인트까지 띄워서 검증 완료
5. ~~기존 pull 스케폴딩 처리~~ → **하이브리드로 둘 다 유지** (\$3.2)
6. ~~인증 방식~~ → **최소 API Key** (\$6.3)
7. ~~배치 부분 실패 처리~~ → **부분 성공 허용** (\$6.4)
8. ~~리뷰 원본 ID 수집 여부~~ → **review_source_id (일련번호만) 추가 수집, 작성자 식별 정보는 계속 미수집** (\$5.2, \$7.3)
9. ~~배치 1건 최대 크기~~ → **100건** (\$6.1)
10. ~~추천 임계값~~ → **10건 이상이면 크롤링 없이 바로 추천** (\$3.3)
11. ~~Query/Recommendation Agent 구현 방식~~ → **분리된 두 단계 호출** (\$3.3, \$4)
12. ~~비동기 전환 시점~~ → **처음(CLI 단계)부터 비동기로 설계** (\$3.3, \$9). ⚠ 다만 \$10.4에서 실제로 만든 1차 구현은 동기다(CLI가 크롤링 끝날 때까지 기다림) — 비동기 알림 방식(\$17)이 아직 없어서 임시로 동기로 연결해둔 상태. 방향 결정을 뒤집은 게 아니라, \$17이 풀리기 전까지의 잠정 구현이다.
13. ~~Query Analysis Agent / Recommendation Agent를 어느 프로세스에 둘 것인가~~ → **Python(CLI와 같은 프로세스, app/services/query_parser.py / recommender.py)에 둬**. 단, LLM 호출이 아니라 규칙 기반 임시 구현이다(ANTHROPIC_API_KEY 없어서). LLM로 교체하는 것은 별도 과제로 남음(\$10.4)

아직 남은 질문

14. 크롤링 실패/부분 성공을 상품 데이터에 플래그로 남길 것인가? (\$9)
15. 재고(품질 여부) 수집 및 stock_history 도입 여부 (\$7.2)
16. PK 전략(AUTO_INCREMENT vs UUID), 인덱스 설계 (\$5.3)
17. Query Analysis Agent / Recommendation Agent를 규칙 기반에서 실제 LLM 호출로 언제 교체할 것인가 (ANTHROPIC_API_KEY 발급 필요) (\$4, \$10.4)
18. CLI 환경에서 비동기 크롤링 완료를 어떻게 알릴 것인가 (폴링 명령/상태 파일 등) (\$3.3, \$9)
19. 자연어 요청 → SQL 조건 매핑 규칙을 정교화할 것인가 (지금은 title LIKE + 가격 이하뿐, 브랜드/할인을 등 조건 추가 필요할지), 임계값(10건)을 카테고리별로 다르게 둘지 (\$3.3, \$9)
20. crawl_target 의 스케줄러를 Spring/Python 중 어디에 둘 것인가, recrawl_interval_minutes 는 어떻게 정할 것인가 (\$7.4)
21. API Key 값 관리 방식(환경변수/설정파일), 검증 계층(Filter/Interceptor) (\$6.3)
22. 배치 분할 호출 중 일부 체크만 실패했을 때 나머지를 계속 보낼지 (\$6.1)
23. 두 번째로 추가할 사이트는 어디로 예정하고 있는가 (구체적인 사이트가 있어야 공통/사이트고유 경계를 제대로 나눌 수 있음) (\$11)
24. "HTML 받기"/"분리(파싱)"를 별도 프로세스로 쪼개야 할 재검토 트리거 기준은 무엇인가 (사이트 수? 물량? 파싱 실패율?) (\$3.1.1)